توضیح کامل و ساده — برای ارائه سر کلاس

کلاس

EventDetailActivity

صفحه جزئیات رویداد — دومین صفحه برنامه

`EventDetailActivity` دومین صفحهٔ برنامهٔ `universityevents` است؛ در این ارائه آن را خط به خط می‌خوانیم:
ساخت صفحه، پیدا کردن عناصر، دریافت Intent، نمایش اطلاعات و بازگشت.

`AppCompatActivity` `Android` `Java`

EventDetailActivity.java

```
public class EventDetailActivity extends AppCompatActivity {
```

این صفحه اطلاعات را نمی‌سازد و تغییر نمی‌دهد؛ فقط نمایش می‌دهد.

بخش ۱

این فایل اصلاً چیست؟

این یک کلاس جاوا به نام `EventDetailActivity` است. اسمش خودش همه چیز را می‌گوید:

Event	Detail	Activity
رویداد	جزئیات	صفحه

یعنی «صفحه جزئیات رویداد»

برای کلاس بگو

«این دومین صفحه برنامه است. وقتی کاربر روی دکمه «مشاهده جزئیات» کلیک می‌کند، این صفحه باز می‌شود و اطلاعات کامل یک رویداد را نشان می‌دهد. این صفحه اطلاعات را نمی‌سازد و تغییر نمی‌دهد؛ فقط نمایش می‌دهد.»

این صفحه از کجا باز می‌شود؟

از `MainActivity`، داخل متد `openEventDetails`:

MainActivity.java

```
Intent intent = new Intent(MainActivity.this,
        EventDetailActivity.class);
startActivity(intent);
```

یعنی «از `MainActivity` برو به `EventDetailActivity`».

بخش ۲

packageها و importها — ابزارهایی که قرض می‌گیریم

EventDetailActivity.java

```
package com.mehrsamweb.universityevents;
import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
```

ابزار	کارش به زبان ساده
Intent	پیک اطلاعات بین صفحه‌ها — اینجا اطلاعات را می‌گیریم (برعکس MainActivity که می‌فرستاد)
Bundle	جعبه اطلاعات اولیه صفحه (اینجا استفاده نمی‌کنیم، ولی onCreate به آن نیاز دارد)
Button	دکمه (برای دکمه «بازگشت»)
TextView	متن (برای نمایش اطلاعات رویداد)
AppCompatActivity	کلاس پدری که می‌گوید «تو یک صفحه‌ای»

importهای این صفحه کمتر از MainActivity است، چون این صفحه کارهای کمتری انجام می‌دهد: نه دیالوگ می‌سازد، نه Spinner دارد، نه اعتبارسنجی — فقط نمایش می‌دهد.

بخش ۳

تعریف کلاس — وارث AppCompatActivity

EventDetailActivity.java

```
public class EventDetailActivity extends AppCompatActivity {
```

همین یک خط، نقش کل صفحه را تعیین می‌کند: این فایل یک صفحهٔ اندرویدی است.

من یک کلاس به نام `EventDetailActivity` می‌سازم که وارث `AppCompatActivity` است.

وارث یعنی چه؟

همه امکانات `AppCompatActivity` را به ارث می‌برد و خودش یک صفحه واقعی می‌شود.

مقایسه با کلاس Event

کلاس `Event` وارث هیچ کلاسی نبود (چون مدل داده بود)، ولی اینجا وارث `AppCompatActivity` است (چون یک صفحه است).

extends

بخش ۴

متد onCreate — لحظه تولد صفحه

EventDetailActivity.java

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_event_detail);
}
```

۱ onCreate

وقتی صفحه اولین بار ساخته می‌شود این متد صدا زده می‌شود — «لحظه تولد این صفحه».

۲ @Override

کاری را که پدرم (AppCompatActivity) قبلاً تعریف کرده بود، دوباره می‌نویسم و شخصی‌سازی می‌کنم.

۳ super.onCreate(savedInstanceState)

اول کارهای پایه پدرم انجام شود، بعد کارهای من — اگر ننویسیم برنامه کرش می‌کند.

۴ setContentView(R.layout.activity_event_detail)

صفحه را با طرح فایل activity_event_detail.xml بساز و نشان بده — R.layout مثل نقشه ساختمان است و setContentView می‌گوید «این نقشه را بساز و تحویل بده».

اسم فایل XML اینجا activity_event_detail است (نه activity_main) — هر Activity فایل XML مخصوص خودش را دارد.

بخش ۵

پیدا کردن عناصر صفحه — findViewById

EventDetailActivity.java

```
TextView txtEventTitle = findViewById(R.id.txtEventTitle);
TextView txtEventDate = findViewById(R.id.txtEventDate);
TextView txtEventLocation = findViewById(R.id.txtEventLocation);
TextView txtEventCategory = findViewById(R.id.txtEventCategory);
TextView txtEventDescription = findViewById(R.id.txtEventDescription);
Button btnBack = findViewById(R.id.btnBack);
```

`findViewById` یعنی «برو در طرحی که ساختی، دنبال عنصری با این شناسه (id) بگرد و آن را به من بده.»

تشبیه: مثل این است که در یک خانه دنبال اتاقی با پلاک «۱۰۳» بگردی.

عنصر	چه چیزی را نشان می‌دهد
`txtEventTitle`	عنوان رویداد
`txtEventDate`	تاریخ
`txtEventLocation`	محل برگزاری
`txtEventCategory`	دسته‌بندی
`txtEventDescription`	توضیحات
`btnBack`	دکمه بازگشت به صفحه قبل

۵ TextView + ۱ Button

پسوند `txt` یعنی «متن» و `btn` یعنی «دکمه» — یک قرارداد نام‌گذاری که خوانایی کد را بالا می‌برد.

بخش ۶

دریافت Intent — مهم‌ترین خط این صفحه

EventDetailActivity.java

```
Intent intent = getIntent();
```

`getIntent()` یعنی «آن پیکی که مرا باز کرد، بده به من.»

تشبیه

فرض کن صفحه قبل یک تاکسی فرستاده که تو را به اینجا آورده. `getIntent()` یعنی «آن تاکسی را بده به من تا ببینم چه چیزی داخلش گذاشته‌اند.»

در MainActivity چه دیدیم؟

MainActivity.java

```
intent.putExtra(EXTRA_EVENT_TITLE, event.getTitle());
startActivity(intent);
```

یعنی «اطلاعات را داخل تاکسی بگذار و راه بینداز» — حالا اینجا با `getIntent()` همان تاکسی را می‌گیریم و اطلاعاتش را بیرون می‌آوریم.

در MainActivity اطلاعات را می‌فرستادیم (`putExtra`)، اینجا اطلاعات را می‌گیریم (`getStringExtra`) — این دو مثل دو سر یک سیم تلفن هستند.

خواندن اطلاعات از `Intent` — `getStringExtra`

بخش ۷

EventDetailActivity.java

```java
String title = intent.getStringExtra(MainActivity.EXTRA_EVENT_TITLE);
String date = intent.getStringExtra(MainActivity.EXTRA_EVENT_DATE);
String location = intent.getStringExtra(MainActivity.EXTRA_EVENT_LOCATION);
String category = intent.getStringExtra(MainActivity.EXTRA_EVENT_CATEGORY);
String description = intent.getStringExtra(MainActivity.EXTRA_EVENT_DESCRIPTION);
```

`getStringExtra` یعنی «از داخل تاکسی، اطلاعاتی که با این برچسب گذاشته شده را بگیر و به من بده.»

در `MainActivity` روی جعبه‌ها برچسب زدیم (`event_title` ، `event_date` و...) — حالا می‌گوییم «آن جعبه‌ای که برچسبش `event_title` است را بده».

چرا ثابت و نه رشته مستقیم؟

getStringExtra()

```java
intent.getStringExtra(MainActivity.EXTRA_EVENT_TITLE)   // درست
intent.getStringExtra("event_title")                    // غلط
```

اگر یک بار دیگر رشته بنویسیم و اشتباه تایپ کنیم (مثلاً `event-titel`)، برنامه خطا نمی‌دهد ولی اطلاعات پیدا نمی‌شود و همه چیز خالی می‌شود!

اصل `DRY` (**Don't Repeat Yourself — خودت را تکرار نکن**): استفاده از ثابت‌های `static final` به جای رشته‌های تکراری جلوی خطای تایپی را می‌گیرد؛ اگر بعداً بخواهیم برچسب را عوض کنیم، فقط یک جا عوض می‌کنیم.

مقدار پیش‌فرض – شبکه ایمنی صفحه

اگر اطلاعاتی از صفحه قبل نرسیده باشد یا خالی باشد، یک **متن پیش‌فرض** جایگزین می‌کنیم تا صفحه هیچ‌وقت خالی یا خراب نمایش داده نشود.

EventDetailActivity.java

```java
if (title == null || title.trim().isEmpty()) {
    title = "عنوان مشخص نشده است";
}
if (date == null || date.trim().isEmpty()) {
    date = "تاریخ مشخص نشده است";
}
// همین الگو برای location ، category و description
```

پنج مقدار پیش‌فرض

`title`	← «عنوان مشخص نشده است»
`date`	← «تاریخ مشخص نشده است»
`location`	← «محل مشخص نشده است»
`category`	← «دسته‌بندی مشخص نشده است»
`description`	← «توضیحی ثبت نشده است»

خواندن شرط، کلمه به کلمه

`title == null`	آیا `title` اصلاً وجود ندارد؟ (چیزی از تاکسی نیامده)
`||`	یا (OR)
`title.trim().isEmpty()`	بعد از حذف فاصله‌های اضافی، خالی است؟

چرا `null`؟ چرا `trim`؟ چرا `||`؟

چرا `null` را چک می‌کنیم؟

`getStringExtra` وقتی برچسب پیدا نشود `null` برمی‌گرداند؛ `null` یعنی «هیچ چیز». اگر مستقیم `title.trim()` را صدا بزنیم و `title` برابر `null` باشد، برنامه کرش می‌کند (`NullPointerException`).

تشبیه: `null` مثل جعبه خالی است — اگر بدون چک کردن درش را باز کنی و انتظار داشته باشی چیزی داخلش باشد، خطا می‌گیری؛ پس اول چک می‌کنیم آیا جعبه وجود دارد یا نه.

چرا `trim().isEmpty()`؟

اگر کاربر در فرم فقط فاصله وارد کرده باشد، این متن از نظر فنی خالی نیست ولی از نظر منطقی هیچ محتوایی ندارد. `trim()` فاصله‌های اضافی را حذف می‌کند و بعد `isEmpty()` چک می‌کند آیا خالی شد یا نه.

چرا `||` درست است؟

اگر هر کدام از دو شرط درست بود، کل شرط درست است. اگر `null` بود، کاری به `isEmpty` نداریم؛ اگر `null` نبود ولی خالی بود، باز هم متن پیش‌فرض می‌گذاریم.

عملگر	معنی	مثال
`||`	یا (OR)	اگر یکی درست باشد، کافی است
`&&`	و (AND)	باید هر دو درست باشند

«این بخش یک **شبکه ایمنی** است. صفحه جزئیات نباید هرگز به کاربر `null` یا متن خالی نشان دهد. حتی اگر صفحه قبل اشتباه کند، این صفحه خودش را نجات می‌دهد — اسم این اصل: `Defensive Programming` (برنامه‌نویسی دفاعی).»

نمایش اطلاعات روی صفحه — setText

بخش ۹

EventDetailActivity.java

```
txtEventTitle.setText(title);
txtEventDate.setText("تاریخ: " + date);
txtEventLocation.setText("محل برگزاری: " + location);
txtEventCategory.setText("دسته‌بندی: " + category);
txtEventDescription.setText("توضیحات: " + description);
```

setText چیست؟

این متن را داخل این `TextView` بگذار و نمایش بده.

تشبیه: مثل این است که یک کاغذ سفید (`TextView`) داری و رویش چیزی می‌نویسی (`setText`).

چرا عنوان بدون پیشوند؟

عنوان خودش واضح است و نیازی به «عنوان:» ندارد؛ برای بقیه، پیشوند («تاریخ: »، «محل برگزاری: » و...) باعث **خوانایی بیشتر** می‌شود.

عملگر + چه کار می‌کند؟

بین دو رشته یعنی «به هم بچسبان» (`Concatenation`):

Concatenation

```
"تاریخ: " + "1405/08/25"
// نتیجه:  تاریخ: 1405/08/25"
```

در جاوا اگر یکی از دو طرف `+` متن باشد یعنی **چسباندن، نه جمع عددی** – اگر هر دو طرف عدد باشند، جمع می‌کند. **این یک نکته مهم است.**

دکمه بازگشت — finish()

بخش ۱۰

EventDetailActivity.java

```
btnBack.setOnClickListener(view → {
    finish();
});
```

setOnClickListener

یعنی «وقتی کاربر روی این دکمه کلیک کرد، این کارها را انجام بده.»

view → { … }

یک لاندا (lambda) است؛ یعنی یک تابع کوچک بدون نام: «ای دکمه! وقتی کلیک شدی، این دستور را اجرا کن.»

finish()

یعنی «این صفحه را ببند و به صفحه قبلی برگرد.» **تشبیه:** مثل این است که یک در را باز کرده باشی و حالا ببندی و برگردی به اتاق قبل.

مقایسه

finish()	startActivity(new Intent(…))
«برو عقب» ←	→ «برو به یک صفحه جدید»
به صفحه‌ای که از آن آمده‌ای	حتی اگر همان صفحه قبلی باشد

اینجا می‌خواهیم برگردیم عقب، پس finish() درست است.

finish() صفحه را **نابود می‌کند** (از حافظه پاک می‌شود)؛ اگر دوباره بخواهیم برگردیم، باید دوباره startActivity کنیم.

ارتباط سه کلاس — زنجیره انتقال اطلاعات

بخش ۱۱

۱ در `MainActivity`

```
Event newEvent = new Event(...);
eventList.add(newEvent);
```

یک شیء `Event` ساخته و نگهداری می‌شود.

۲ کلیک کاربر

```
openEventDetails(event);
```

کاربر روی «مشاهده جزئیات» کلیک می‌کند.

۳ در `openEventDetails` (داخل MainActivity)

`Intent` ساخته می‌شود؛ اطلاعات از شیء `Event` خوانده، با `putExtra` داخل `Intent` گذاشته و با `startActivity` به صفحه بعد فرستاده می‌شود.

۴ در `EventDetailActivity` (همین فایل)

با `getIntent()` تاکسی گرفته می‌شود، با `getStringExtra` اطلاعات خوانده و با `setText` روی صفحه نمایش داده می‌شود.

خلاصه جریان اطلاعات (جهت خواندن: از راست به چپ)



«`Event` اطلاعات را نگه می‌دارد. `MainActivity` اطلاعات را از `Event` می‌خواند و داخل `Intent` می‌گذارد. `EventDetailActivity` اطلاعات را از `Intent` می‌خواند و روی صفحه نشان می‌دهد.
این یک زنجیره است.»

بخش ۱۲ · مقایسه دو صفحه

چرا این صفحه خیلی ساده‌تر از `MainActivity` است؟

`MainActivity` در برابر `EventDetailActivity`

ویژگی	MainActivity	EventDetailActivity
نقش	صفحه اصلی + مدیریت لیست	فقط نمایش جزئیات
آیا داده می‌سازد؟	بله (فرم افزودن)	نه
آیا داده تغییر می‌دهد؟	بله (حذف)	نه
آیا دیالوگ دارد؟	بله (فرم و تأیید حذف)	نه
آیا اعتبارسنجی دارد؟	بله	فقط چک `null` و خالی
آیا لیست دارد؟	بله	نه
ورودی از کجا؟	کاربر	`Intent`
خروجی به کجا؟	`Intent` + نمایش	فقط نمایش

«این صفحه یک صفحه فقط-خواندنی است. کاربر فقط می‌تواند ببیند و برگردد؛ هیچ چیزی را نمی‌تواند تغییر دهد. **اصل مهم: هر صفحه یک کار انجام دهد.**»

بخش ۱۳ · جمع‌بندی

۲ دقیقه · ۸ گام

جمع‌بندی سریع — کل صفحه در ۲ دقیقه

۱

این صفحه جزئیات یک رویداد است.

۲

در `onCreate` صفحه ساخته می‌شود و عناصر (۵ متن + ۱ دکمه) پیدا می‌شوند.

۳

با `getIntent()` همان تاکسی را می‌گیریم که از `MainActivity` آمده.

۴

با `getStringExtra` پنج اطلاعات رویداد را از تاکسی بیرون می‌آوریم.

۵

اگر اطلاعاتی نرسیده باشد یا خالی باشد، متن پیش‌فرض می‌گذاریم تا صفحه خراب نشود.

۶

با `setText` اطلاعات را روی صفحه نمایش می‌دهیم.

۷

با کلیک روی دکمه بازگشت، `finish()` صدا زده می‌شود و به صفحه قبل برمی‌گردیم.

۸

این صفحه فقط نمایش می‌دهد؛ هیچ چیزی نمی‌سازد و تغییر نمی‌دهد.

کل چرخه صفحه در ۸ گام: از لحظه ساخته‌شدن در `onCreate` تا بسته‌شدن با `finish()` — بدون ساخت و بدون تغییر داده؛ این صفحه فقط نمایش می‌دهد.

۵ پرسش

چند سؤال احتمالی — و جواب کوتاه

س: `getIntent()` چیست؟

ج: «آن `Intent` که این صفحه را باز کرده، برگردان.» مثل گرفتن تاکسی‌ای که تو را به اینجا آورده.

س: تفاوت `putExtra` و `getStringExtra` چیست؟

ج: `putExtra` = گذاشتن اطلاعات در `Intent` (در `MainActivity`).
`getStringExtra` = گرفتن اطلاعات از `Intent` (در `EventDetailActivity`).

س: چرا `null` را چک می‌کنیم؟

ج: چون `getStringExtra` اگر برچسب را پیدا نکند `null` برمی‌گرداند؛ `null` یعنی «هیچ چیز». اگر بدون چک کردن استفاده کنیم، برنامه کرش می‌کند.

س: چرا `trim()` و `isEmpty()` را با هم چک می‌کنیم؟

ج: `trim()` فاصله‌های اضافی را حذف می‌کند، `isEmpty()` چک می‌کند خالی است یا نه؛ با هم یعنی «اگر بعد از حذف فاصله‌ها خالی بود».

س: `||` چه فرقی با `&&` دارد؟ **ج:** `||` = یا (یکی کافی است). `&&` = و (هر دو لازم است).

چند سؤال احتمالی — و جواب کوتاه

۴ پرسش

۱ س: `finish()` چه کار می‌کند؟

ج: این صفحه را می‌بندد و به صفحه قبل برمی‌گردد.

۲ س: چرا از `MainActivity.EXTRA_EVENT_TITLE` استفاده کردیم و نه `"event_title"`؟

ج: چون ثابت است و جلوی خطای تایپی را می‌گیرد؛ اگر بعداً بخواهیم برچسب را عوض کنیم، فقط یک جا عوض می‌کنیم.

۳ س: چرا این صفحه `Event` را مستقیم نمی‌گیرد؟

ج: چون `Intent` فقط انواع ساده (متن، عدد) را می‌شناسد. برای فرستادن شیء کامل باید `Event` را `Serializable` یا `Parcelable` کنیم که در این جلسه انجام ندادیم؛ پس فعلاً تک‌تک فیلدها را می‌فرستیم و می‌گیریم.

۴ س: چرا این صفحه اطلاعات را تغییر نمی‌دهد؟

ج: چون نقشش فقط نمایش است. اگر بخواهیم ویرایش کنیم، باید صفحه جدید یا دکمه ویرایش اضافه کنیم (برای جلسات بعد).

مقایسه نهایی سه کلاس برنامه (برای درک کلی)

ردیف	Event	MainActivity	EventDetailActivity
نقش	مدل داده	صفحه اصلی	صفحه جزئیات
وارث	هیچ کلاسی	`AppCompatActivity`	`AppCompatActivity`
کار	نگهداری اطلاعات	ساخت، حذف، نمایش لیست	نمایش جزئیات
ورودی	سازنده	کاربر (کلیک، فرم)	`Intent`
خروجی	`Getter/Setter`	`Intent` + نمایش	فقط نمایش
پیچیدگی	کم	زیاد	کم

این سه کلاس با هم یک برنامه کامل می‌سازند.

`Event` قلب است (اطلاعات).

`MainActivity` دست و چشم است (ساخت و نمایش).

`EventDetailActivity` ذره‌بین است (نمایش دقیق‌تر).

هر کدام یک کار، و با هم یک برنامه.

پایان ارائه

با تشکر از توجه شما

`EventDetailActivity` — صفحه‌ای که فقط نمایش می‌دهد.

هر صفحه یک کار انجام دهد | Defensive Programming | DRY — خودت را تکرار نکن

EventDetailActivity.java

```
btnBack.setOnClickListener(view → { finish(); });
```

آیا سوالی هست؟ — مثل همین صفحه، منتظر پرسش‌های شما هستیم!